

LCD display module

The project uses a Hitachi HD44780-controlled LCD module. The HD44780 controller requires three control lines and four or eight input/output (I/O) lines for the data bus. The user may choose to operate the LCD with a 4-bit or 8-bit data bus. If a 4-bit data bus is used, the LCD will require a total of seven data lines—three lines for sending control signals to the LCD and four lines for the data bus. If an 8-bit data bus is used, the LCD will require a total of eleven data lines—three control lines and eight lines for the data bus.

The enable control line is used to tell the LCD that the microcontroller is sending the data to it. To send data to the LCD, first make sure that the enable line is low (0). When other control lines are completely ready, make enable pin high and wait for the LCD to be ready. This time is mentioned in the datasheet and varies from LCD to LCD. To stop sending the data, bring the enable control low (0) again. Fig. 2 shows the timing diagram of LCD control lines for 4-bit data during write operation.

When the register-select line is low (0), the data is treated as a command or special instruction (such as clear screen and position cursor). When register-select is high (1), the text data being sent is displayed on the screen. For example, to display letter 'L' on the screen, register-select is set to high.

When the read/write control line is low, the information on the data bus is written to the LCD. When read/write is high, the program effectively queries (or reads) the LCD. This control command can be implemented using 'C' programming language.

For 4-bit interface data, only four bus lines (D4 through D7) are used for data transfer. Bus lines D0 through D3 are disabled. The data transfer between HD44780 and the microcontroller completes after the 4-bit data is transferred twice.

Controlling a standard numeric

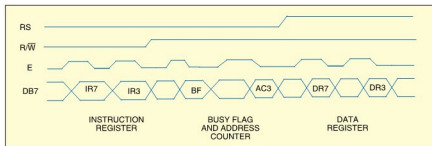


Fig. 2: Timing diagram of LCD control lines for 4-bit data during write operation

LCD is not that difficult. To display text on the LCD, correct library files for the LCD are needed. Many LCD libraries are available on the Internet, which are used in various applications. You may get confused which library is suitable for your application.

Libraries for LCDs found in AVR-LIB library occupy unnecessary program memory space. To solve the problem, you can write your own library for LCD control.

Software program

This project demonstrates sending the text to the LCD controller and scrolling it across the LCD. For the project, AVR Studio 4 and WINAVR software need to be installed in your PC. Three program codes are used here—movm.c, lcd2.c and lcd2.h. The movm.c contains the text message to be scrolled on the LCD. lcd2.c and lcd2.h are the library files. The programming technique given here may not be the best as it uses a simple logic, but it works pretty fine.

The LCD library for 4-line or 4-bit mode operation is used here. Each pin connected to the LCD can be defined separately in the lcd2h code. The LCD and AVR port configurations in the C code along with comments are given below:

```
#define LCD_RS_PORT LCD_PORT
LCD port for RS line
#define LCD_RS_PIN 2
PORTC bit 2 for RS line
#define LCD_RW_PORT PORTD
Port for RW line
#define LCD_RW_PIN 6
PORTD bit 6 for RW line
#define LCD_E_PORT LCD_PORT
LCD port for enable line
```

```
#define LCD_E_PIN 3
```

PORTC bit 3 for enable line

Enable control line. The E control line is used to tell the LCD that the instruction for sending the data on the data bus is ready to be executed. E must always be manipulated when communicating with the LCD. That is, before interacting with the LCD, E line is always made low. The following instructions toggle enable pin to initiate write operation:

```
/* toggle enable pin to initiate
write */
static void toggle_e(void)
{
    lcd_e_high();
    lcd_e_delay();
    lcd_e_low();
}
```

The complete subroutine of this code can be found in lcd2.c.

The E line must be left high for the time required by the LCD to get ready for receiving the data; it's normally about 250 nanoseconds (check the datasheet for exact duration).

Busy status of the LCD. It takes some time for each instruction to be executed by the LCD. The delay varies depending on the frequency of the crystal attached to the oscillator input of the HD44780 as well as the instruction being executed.

While it is possible to write the code that waits for a specific amount of time to allow the LCD to execute instructions, this method of waiting is not very flexible. If the crystal frequency is changed, the software needs to be modified. Additionally, if the LCD itself is changed, the program might not work even if the new LCD is